R code for: A nearly 1.5 millennia long record of North Atlantic climatic forcing recorded tree-ring growth records from Angstel/Vecht Delta, the Netherlands (1283 BCE – 156 CE)

Michiel Arts[1] Jos Bazelmans[2,3]
1 - SEDICLIM lab, Department of Geology, University of Liege, Belgium
2 - Cultural Heritage Agency of the Netherlands and Faculty of Archaeology
3 - University of Leiden, Leiden, Netherlands

2026-04-01

Load data

```
library(readxl)
library(astrochron)
library(WaverideR)
library(raster)
library(jpeg)
library(httr)
library(gt)

# analysis of the 3rd column of the Angstel Vecht dataset #####
setwd("D:/Phd/documents/Weesp")
VA25_1 <- read_excel("VA25_1.xlsx")
VA25_1 <- as.data.frame(VA25_1)
# graphics.off()
# plot(VA25_1[, 1],
#      VA25_1[, 3],
#      type = "l",
#      xlab = "year",
```

```
#      ylab = "VA25")
# par(new = TRUE)
VA25mean <- cbind(VA25_1[, 1], VA25_1[, 3])
VA25mean <- iso(VA25mean,xmin=-1283,xmax=156,genplot=FALSE)

#load the baker 2015 dataset
stal_scot <- read.csv("stal_scot.csv")
```

Load paleogeographic map without Weesp bog oak locations

```
# URL of the raw image
url <- "https://raw.githubusercontent.com/stratigraphy/Weesp-dendro-cycles/main/palgeo.jpg"

# Download into memory
res <- GET(url)
stop_for_status(res)

# Read the JPEG from the raw content
img <- readJPEG(content(res, "raw"))

par(mar = c(4, 2, 2, 2))

# Empty plot to define margins and space in the panel
plot(1, type = "n", axes = FALSE, xlab = "", ylab = "", main="")

# Get the coordinates of the current plot region
usr <- par("usr")  # c(x1, x2, y1, y2)

# Draw the image to fill the entire panel
rasterImage(img, usr[1], usr[3], usr[2], usr[4])
```

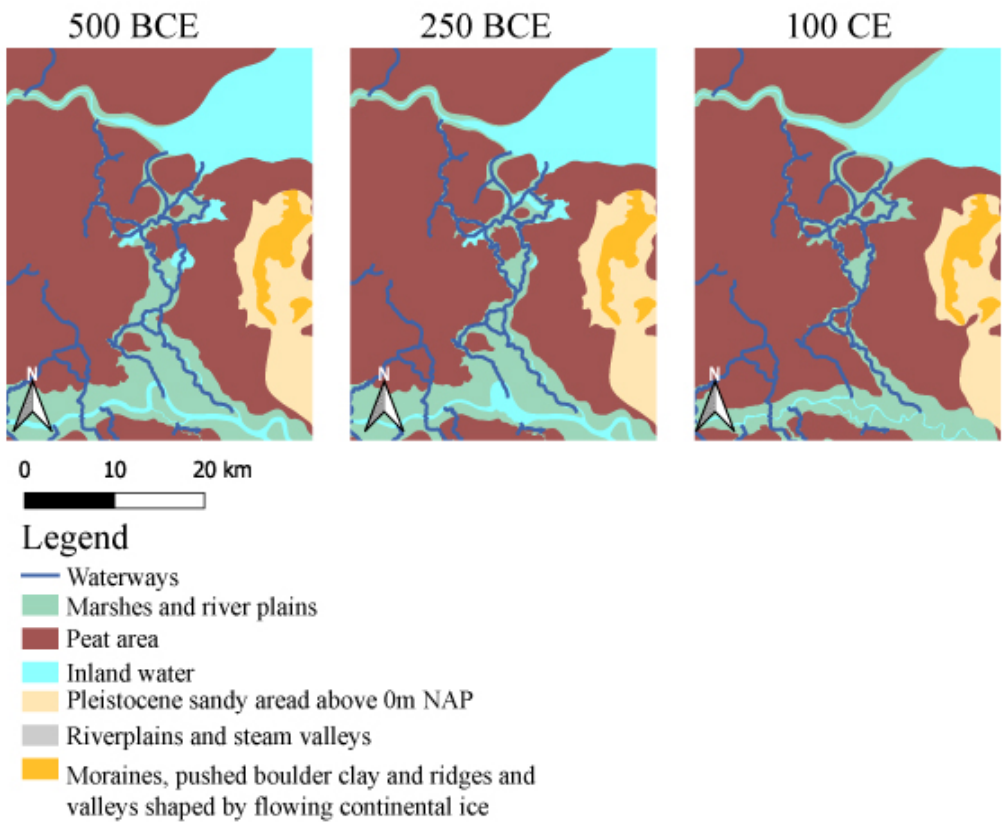

Figure 1: Figure 1. Paleogeographic overview of the Angstel/Vecht delta modified after Vos et al., (2020) (sampling locatilies to be added)

```
par(mar = c(4, 6, 2, 2))
```

Plot data

```
# Two plots stacked vertically
par(mfrow = c(2, 1),
    mar = c(4, 4, 2, 1),
```

```r
    oma = c(0, 0, 2, 0))
# Define year range
years <- VA25_1[, 1]
VA25_1 <- VA25_1[VA25_1[,1] >= -1283 & VA25_1[,1] <= 156, ]
xlim <-range(VA25_1[,1] )


# 1. Create the continuous plotting year variable
# Shifts -1 to 0 so the gap between -1 and 1 is exactly 1 unit
VA25_1$PlotYear <- ifelse(VA25_1[, 1] < 0, VA25_1[, 1] + 1, VA25_1[, 1])

# Setup plot area
par(mfrow = c(2, 1),
    mar = c(4, 4, 2, 1),
    oma = c(0, 0, 2, 0))

xlim_plot <- range(VA25_1$PlotYear)

add_custom_axis <- function(xlim_plot) {
  # 1. Define the PRETTY labels (rounded numbers)
  pretty_labels <- seq(-1250, 250, by = 250)

  # Remove 0, 1, and -1 from the rounded list so they don't overlap our specific '1'
  pretty_labels <- pretty_labels[!(pretty_labels %in% c(-1, 0, 1))]

  # 2. Map those labels to the PLOT positions (BCE moves +1)
  pretty_at <- ifelse(pretty_labels < 0, pretty_labels + 1, pretty_labels)

  # 3. Define only '1' as the special transition point
  # Year 1 sits at position 1 on our x-axis
  special_label <- "1"
  special_at <- 1

  # 4. Define minor ticks (mapped to plot scale)
  minor_years <- seq(-1300, 200, by = 50)
  minor_at <- ifelse(minor_years < 0, minor_years + 1, minor_years)

  # Draw minor ticks
  axis(1, at = minor_at, labels = FALSE, tck = -0.02, col = "black")

  # Draw the pretty major ticks (e.g., -1250, -1000...)
  axis(1, at = pretty_at, labels = pretty_labels, tck = -0.05)
```

```r
  # Draw the specific '1' tick
  axis(1, at = special_at, labels = special_label, tck = -0.05, font = 2)
}

# First plot (blue)
plot(VA25_1$PlotYear, VA25_1[, 2],
     type = "l", col = "blue",
     xlab = "Year", ylab = "Width (mm)",
     main = "Mean ring width", xaxt = "n")
add_custom_axis(xlim_plot)

# Second plot (red)
plot(VA25_1$PlotYear, VA25_1[, 3],
     type = "l", col = "red",
     xlab = "Year", ylab = "Index",
     main = "Growth Index (GI)", xaxt = "n")
add_custom_axis(xlim_plot)
```

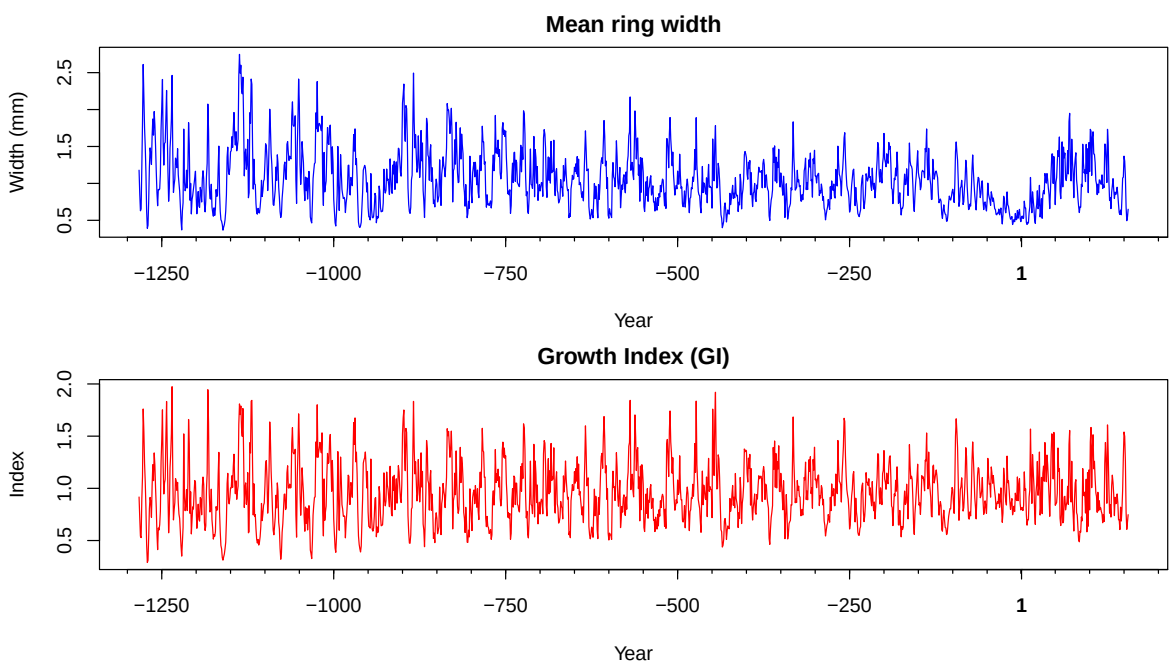

Figure 2: Figure 2. Tree ring growth rate data set of Bazelmans et al., (2025)

Wavelet scalogram

```
VA25mean <- linterp(VA25mean,genplot = FALSE,verbose=FALSE)
VA25mean <- sortNave(VA25mean,genplot = FALSE,verbose=FALSE)

VA25mean_wt <- analyze_wavelet(
  data = VA25mean,
  dj = 1 / 100,
  lowerPeriod = 1,
  upperPeriod = 256,
  verbose = FALSE,
  omega_nr = 10,
  pval = FALSE,
  n_simulations = 10,
  run_multicore = FALSE
)

#The BCE_CE_wavelet_plot function is a modified version of the plot_wavelet function from the

BCE_CE_wavelet_plot <- function (wavelet = NULL, lowerPeriod = NULL, upperPeriod = NULL,
          n.levels = 100, palette_name = "rainbow", color_brewer = "grDevices",
          useRaster = TRUE, periodlab = "Period (metres)", x_lab = "depth (metres)",
          keep_editable = FALSE, dev_new = TRUE, plot_dir = TRUE,
          add_lines = NULL, add_points = NULL, add_abline_h = NULL,
          add_abline_v = NULL, add_MTM_peaks = FALSE, add_data = TRUE,
          add_avg = FALSE, add_pval = FALSE, pval_abline = c(0.1,
                                                             0.05), pval_cutoff = c(0.1), add_
          demean_mtm = TRUE, detrend_mtm = TRUE, padfac_mtm = 5, tbw_mtm = 3,
          plot_horizontal = TRUE)
{
  if (keep_editable == FALSE) {
    oldpar <- par(no.readonly = TRUE)
    on.exit(par(oldpar))
  }

  # Data is already adapted (Year -1 = 0), so we use wavelet$x directly
  plot_x <- as.numeric(wavelet$x)

  maximum.level = max(wavelet$Power)
  power_max_mat.levels = quantile(wavelet$Power, probs = seq(from = 0,
                                                             to = 1, length.out = n.levels +
```

```
if (color_brewer == "RColorBrewer") {
  key.cols <- rev(colorRampPalette(brewer.pal(brewer.pal.info[palette_name, 1], palette_nar
}
if (color_brewer == "colorRamps") {
  color_brewer_Sel <- paste("colorRamps::", palette_name, "(n=n.levels)")
  key.cols = eval(parse(text = color_brewer_Sel))
}
if (color_brewer == "grDevices") {
  if (palette_name == "rainbow") {
    color_brewer_Sel <- "grDevices::rainbow(n=n.levels, start = 0, end = 0.7)"
    key.cols <- rev(eval(parse(text = color_brewer_Sel)))
  }
  else if (palette_name == "heat.colors" | palette_name == "terrain.colors" |
           palette_name == "topo.colors" | palette_name == "cm.colors") {
    color_brewer_Sel <- paste("grDevices::", palette_name, "(n=n.levels, start = 0, end = 1
    key.cols <- rev(eval(parse(text = color_brewer_Sel)))
  }
  else {
    key.cols <- hcl.colors(n = n.levels, palette = palette_name, alpha = NULL, rev = FALSE
  }
}
if (color_brewer == "viridis") {
  color_brewer_Sel <- paste("viridis::", palette_name, "(n=n.levels,direction = -1)")
  key.cols = rev(eval(parse(text = color_brewer_Sel)))
}
if (color_brewer == "scico") {
  color_brewer_Sel <- paste("scico::scico(n = n.levels, palette = '", palette_name, "', dir
  key.cols = rev(eval(parse(text = color_brewer_Sel)))
}

periodtck = 0.02
periodtcl = 0.5
main = NULL
lwd.axis = 1
legend.params = list(width = 1.2, shrink = 0.9, mar = 5.1, n.ticks = 6, label.digits = 3, l
key.marks = round(seq(from = 0, to = 1, length.out = legend.params$n.ticks) * n.levels)
key.labels = formatC(as.numeric(power_max_mat.levels), digits = legend.params$label.digits

if (dev_new == TRUE) {
  width_val = ifelse(plot_horizontal, 15, 7)
  height_val = ifelse(plot_horizontal, 7, 10)
  dev.new(width = width_val, height = height_val, noRStudioGD = TRUE)
```

```
}

pmax_avg_sel <- t(wavelet$Power)
y_axis <- as.numeric(wavelet$Period)
MTM_res_2 <- matrix(data = NA, ncol = 2, nrow = 1)

xlim_vals = if (plot_dir != TRUE) rev(range(plot_x)) else range(plot_x)
lowerPeriod <- if (is.null(lowerPeriod)) min(wavelet$Period) else lowerPeriod
upperPeriod <- if (is.null(upperPeriod)) max(wavelet$Period) else upperPeriod
ylim_vals = c(lowerPeriod, upperPeriod)

if (add_data == TRUE && add_avg == TRUE && plot_horizontal == TRUE) {
  layout.matrix <- matrix(c(1, 2, 4, 3), nrow = 2, ncol = 2, byrow = TRUE)
  graphics::layout(mat = layout.matrix, heights = c(0.25, 1), widths = c(8, 2))

  # Top Plot (Proxy Data)
  par(mar = c(0, 4, 2, 0))
  plot(y = wavelet$y, x = plot_x, type = "l", xaxs = "i", xlab = "", ylab = "proxy value",
  if (!is.null(add_abline_v)) abline(v = add_abline_v)

  # Legend
  par(new = FALSE, mar = c(3, 2, 2, 2), mgp = c(2, 1, 0))
  image(x = seq(from = 0, to = n.levels), y = 1, z = t(matrix(power_max_mat.levels, nrow =
  axis(1, lwd = lwd.axis, at = key.marks, labels = NA, tck = 0.02, tcl = 1.2)
  mtext(key.labels, side = 1, at = key.marks, line = 0.1, las = 2, cex = 0.75)
  box(lwd = lwd.axis)

  # Side Plot (Average Power)
  par(new = FALSE, mar = c(4, 0, 0, 0.5))
  plot(x = wavelet$Power.avg, y = wavelet$Period, log = "y", type = "l", yaxs = "i", yaxt =
  if (!is.null(add_abline_h)) abline(h = add_abline_h)

  # Main Wavelet Image
  par(new = FALSE, mar = c(4, 4, 0, 0), xpd = FALSE)
  image(x = plot_x, y = wavelet$axis.2, z = t(wavelet$Power), col = key.cols, breaks = powe

  # --- BCE/CE AXIS LOGIC ---
  pretty_labels <- seq(floor(min(plot_x-1)/250)*250, ceiling(max(plot_x)/250)*250, by = 250
  pretty_labels <- pretty_labels[!(pretty_labels %in% c(-1, 0, 1))]
  pretty_at <- ifelse(pretty_labels < 0, pretty_labels + 1, pretty_labels)

  axis(1, at = pretty_at, labels = pretty_labels)
```

```r
    axis(1, at = 1, labels = "1", font = 2)
    # --------------------------

    # COI (Using original wavelet coordinates which match plot_x length)
    polygon(x = wavelet$coi.1, y = wavelet$coi.2, border = NA, col = rgb(1, 1, 1, 0.5))
    abline(h = log2(add_abline_h))
    box(lwd = lwd.axis)
    period.tick = na.omit(unique(trunc(wavelet$axis.2)))
    axis(2, lwd = lwd.axis, at = period.tick, labels = 2^period.tick, las = 2, tck = periodt
  }
}




BCE_CE_wavelet_plot(
  VA25mean_wt,
  add_avg = TRUE,
  add_abline_h = c(2.2, 3.6,5.9, 8,12, 15.4,24, 38, 60,80, 110, 200),
  x_lab = "Age year",
  periodlab = "Period years",
  add_MTM = FALSE,dev_new = FALSE,
  keep_editable = TRUE
)
```

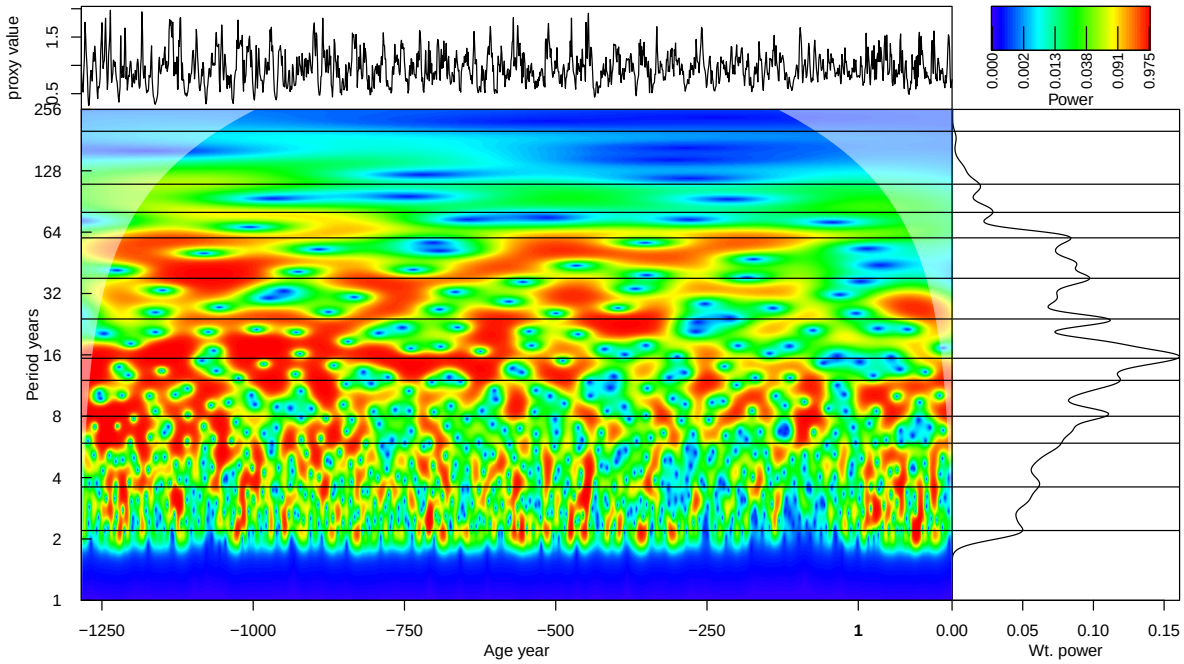

Figure 3: Figure 3. The Growth rate index record and its wavelet scalogram, indicated with black horizontal lines are spectral peaks in the average wavelet power scalogram

Extract cycles and their spectral power

```
VA25mean_3.4 <- taner(
  VA25mean,
  flow = 1 / (3.4 + 1.6),
  fhigh = 1 / (3.4 - 1.4),
  roll = 10 ^ 10,
  xmax = 1 / 50,
  detrend = FALSE,genplot=FALSE
)

VA25mean_8 <- taner(
  VA25mean,
  flow = 1 / (8 + 2),
  fhigh = 1 / (8 - 2),
  roll = 10 ^ 20,
```

```r
  xmax = 1 / 5,
  detrend = FALSE,genplot=FALSE
)

VA25mean_15 <- taner(
  VA25mean,
  flow = 1 / (15 + 4),
  fhigh = 1 / (15 - 3),
  roll = 10 ^ 20,
  xmax = 1 / 5,
  detrend = FALSE,genplot=FALSE)



VA25mean_35 <- taner(
  VA25mean,
  flow = 1 / (38 + 6.5),
  fhigh = 1 / (38 - 6.5),
  roll = 10 ^ 20,
  xmax = 1 / 15,
  detrend = FALSE,genplot=FALSE
)


VA25mean_60 <- taner(
  VA25mean,
  flow = 1 / (60 + 7),
  fhigh = 1 / (60 - 10),
  roll = 10 ^ 20,
  xmax = 1 / 30,
  detrend = FALSE,genplot=FALSE
)

VA25mean_120 <- taner(
  VA25mean,
  flow = 1 / (110 + 17.5),
  fhigh = 1 / (110 - 17.5),
  roll = 10 ^ 10,
  xmax = 1 / 50,
  detrend = FALSE,genplot=FALSE
)
```

```r
pwr_8 <- extract_power_stable(
  scalogram = VA25mean_wt,
  cycle = 8,
  period_up = 1.2,
  period_down = 0.8
)

pwr_3.4 <- extract_power_stable(
  scalogram = VA25mean_wt,
  cycle = 3.4,
  period_up = 1.2,
  period_down = 0.8
)
pwr_3_4 <- pwr_3.4
pwr_16 <- extract_power_stable(
  scalogram = VA25mean_wt,
  cycle = 15,
  period_up = 1.2,
  period_down = 0.8
)

pwr_35 <- extract_power_stable(
  scalogram = VA25mean_wt,
  cycle = 38,
  period_up = 1.2,
  period_down = 0.8
)

pwr_60 <- extract_power_stable(
  scalogram = VA25mean_wt,
  cycle = 60,
  period_up = 1.2,
  period_down = 0.8
)

pwr_120 <- extract_power_stable(
  scalogram = VA25mean_wt,
  cycle = 110,
  period_up = 1.2,
  period_down = 0.8
)
```

```r
# --- Define pastel color palette ---
cols <- c(
  "110 kyr" = "#f7686c",
  "60 kyr"  = "#6baed6",
  "38 kyr"  = "#74c476",
  "15 kyr"  = "#9e9ac8",
  "8 kyr"   = "#fd8d3c",
  "3.4 kyr" = "#fff34d"
)

# --- Layout setup ---
layout.matrix <- matrix(1:7, nrow = 7, ncol = 1)
layout(mat = layout.matrix,
       heights = c(1,1,1,1,1,1,4),
       widths = 1)
par(mar = c(0, 5, 1, 1))

# --- Helper function for subplot labels ---
add_label <- function(label) {
  usr <- par("usr")
  text(x = usr[1] + 0.02 * (usr[2] - usr[1]),
       y = usr[4] - 0.075 * (usr[4] - usr[3]),
       labels = label,
       font = 2,
       cex = 1.2)
}

# --- 1. ~110 kyr cycle (Panel A) ---
plot(VA25mean, type = "l", xlab = "", ylab = "GI",
     main = "110 yr cycle", col = "black", xaxt = "n")
lines(VA25mean_120, col = cols["110 kyr"], lwd = 2)
add_label("A")

# --- 2. ~60 kyr cycle (Panel B) ---
plot(VA25mean, type = "l", xlab = "", ylab = "GI",
     main = "60 yr cycle", col = "black", xaxt = "n")
lines(VA25mean_60, col = cols["60 kyr"], lwd = 2)
add_label("B")

# --- 3. ~38 kyr cycle (Panel C) ---
plot(VA25mean, type = "l", xlab = "", ylab = "GI",
     main = "38 yr cycle", col = "black", xaxt = "n")
```

```r
lines(VA25mean_35, col = cols["38 kyr"], lwd = 2)
add_label("C")

# --- 4. ~15 kyr cycle (Panel D) ---
plot(VA25mean, type = "l", xlab = "", ylab = "GI",
     main = "15 yr cycle", col = "black", xaxt = "n")
lines(VA25mean_15, col = cols["15 kyr"], lwd = 2)
add_label("D")

# --- 5. ~8 kyr cycle (Panel E) ---
plot(VA25mean, type = "l", xlab = "", ylab = "GI",
     main = "8 yr cycle", col = "black", xaxt = "n")
lines(VA25mean_8, col = cols["8 kyr"], lwd = 2)
add_label("E")

# --- 6. ~3.4 kyr cycle (Panel F) ---
plot(VA25mean, type = "l", xlab = "", ylab = "GI",
     main = "3.4 yr cycle", col = "black", xaxt = "n")
lines(VA25mean_3.4, col = cols["3.4 kyr"], lwd = 2)
add_label("F")

# --- 7. Stacked spectral power polygons (Panel G) ---
par(mar = c(4, 5, 1, 1))
# Use pwr_8[,1] which contains the adapted plot positions
plot_x_range <- range(pwr_8[, 1])
plot(0, 0, type = "n", xlim = plot_x_range, ylim = c(0, 100),
     xlab = "year", ylab = "Percentage spectral power (%)",
     main = "Percentage spectral power contribution", xaxt = "n")

# --- Custom BCE/CE Axis Adjustment ---
pretty_labels <- seq(-1250, 250, by = 250)
pretty_labels <- pretty_labels[!(pretty_labels %in% c(-1, 0, 1))]
# Map labels to plotting positions (BCE moves +1)
pretty_at <- ifelse(pretty_labels < 0, pretty_labels + 1, pretty_labels)

axis(1, at = pretty_at, labels = pretty_labels)
axis(1, at = 1, labels = "1", font = 2)
# --------------------------------------

add_label("G")

# Stacked Polygons (Total sum down to individual components)
```

```
x_poly <- pwr_8[,1]

# 110 kyr (Total stack)
polygon(x = c(x_poly, rev(x_poly)),
        y = c((pwr_120[,4] + pwr_60[,4] + pwr_35[,4] + pwr_16[,4] + pwr_8[,4] + pwr_3_4[,4])*
        col = cols["110 kyr"], border = NA)

# 60 kyr
polygon(x = c(x_poly, rev(x_poly)),
        y = c((pwr_60[,4] + pwr_35[,4] + pwr_16[,4] + pwr_8[,4] + pwr_3_4[,4])*100, rep(0, le
        col = cols["60 kyr"], border = NA)

# 38 kyr
polygon(x = c(x_poly, rev(x_poly)),
        y = c((pwr_35[,4] + pwr_16[,4] + pwr_8[,4] + pwr_3_4[,4])*100, rep(0, length(x_poly)
        col = cols["38 kyr"], border = NA)

# 15 kyr
polygon(x = c(x_poly, rev(x_poly)),
        y = c((pwr_16[,4] + pwr_8[,4] + pwr_3_4[,4])*100, rep(0, length(x_poly))),
        col = cols["15 kyr"], border = NA)

# 8 kyr
polygon(x = c(x_poly, rev(x_poly)),
        y = c((pwr_8[,4] + pwr_3_4[,4])*100, rep(0, length(x_poly))),
        col = cols["8 kyr"], border = NA)

# 3.4 kyr
polygon(x = c(x_poly, rev(x_poly)),
        y = c(pwr_3_4[,4]*100, rep(0, length(x_poly))),
        col = cols["3.4 kyr"], border = NA)

legend("topright", legend = names(cols), fill = cols, border = NA, bty = "n", cex = 0.9, titl
```

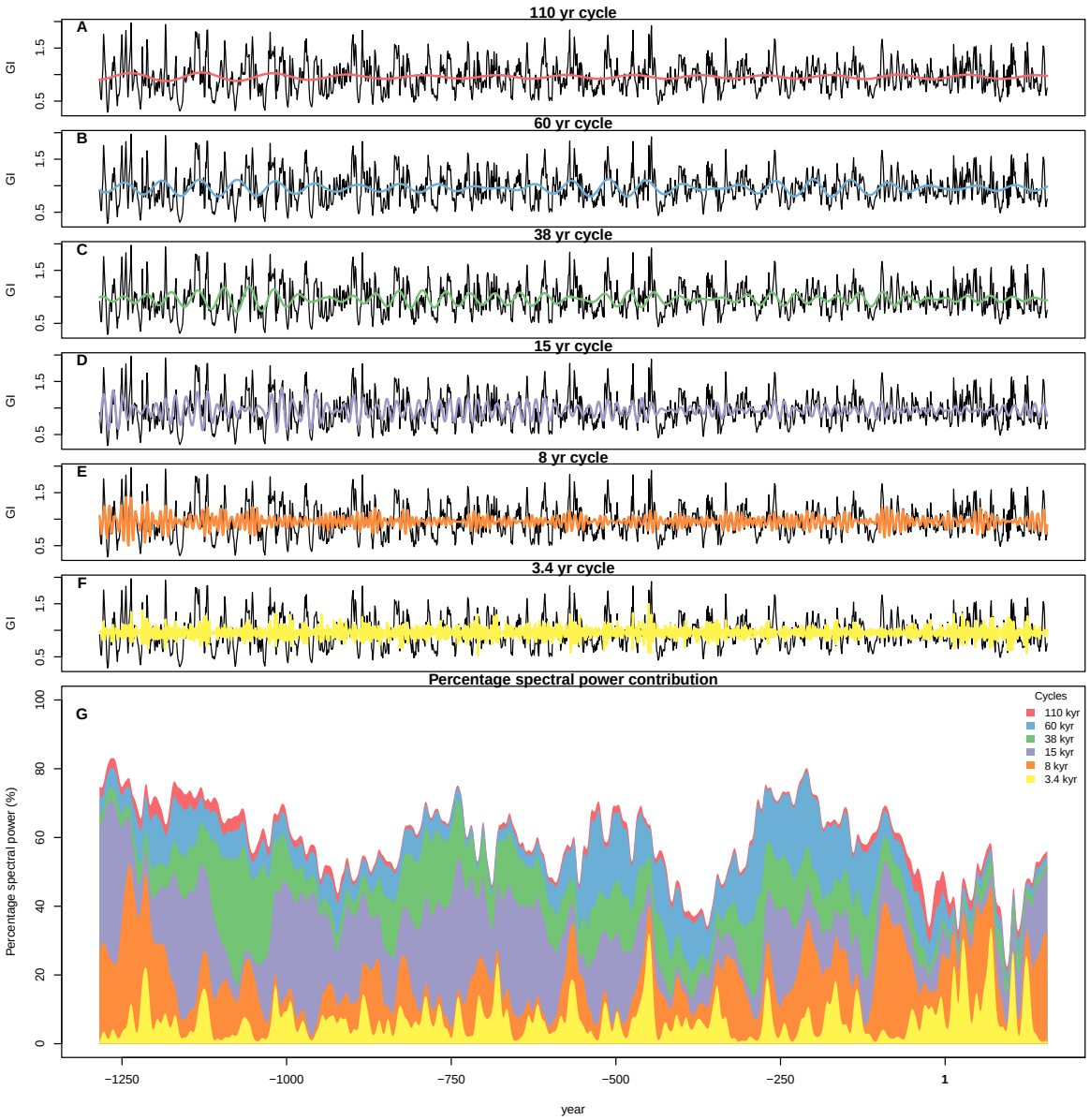

Figure 4: Figure 4. A. Growth Index record and 110 yr cycle on top. B. Growth Index record and 60 yr cycle on top. C. Growth Index record and 38 yr cycle on top. D. Growth Index record and 15 yr cycle on top. E. Growth Index record and 8 yr cycle on top. F. Growth Index record and 3.4 yr cycle on top. G contribution of the 3.4, 8,15,38,60 and 110 yr cycles to the total spectral power

Bicoherence Analysis

```
library(fields)

#The code blow is modified version of the biocherence function from the Astrochron R package

periods <- c(1/5.7,1/6.9,1/8,1/9.5,1/10.75,1/12,1/14,1/16,1/18,1/24, 1/40, 1/60,1/100)

dat=VA25mean
overlap = 100
segments = 8
CF = 0
CL = 90
padfac = 50
demean = T
detrend =  T
taper = T
maxF = 1 / 5
output = 1
genplot = T
color = 1
id = NULL
logpwr = T
logbis = T
check = T
verbose = T
normfft = T
if (color == 0)
  colscale = gray((0:499) / 499)
if (color == 1)
  colscale = hcl.colors(12, "YlOrRd", rev = TRUE)
ply = ""
plx = ""
if (logpwr) {
  ply = "y"
  plx = "x"
}



d <- data.frame(dat)
```

```
npts <- length(d[, 1])
dt = abs(d[2, 1] - d[1, 1])
if (check) {
  dtest <- d[2:npts, 1] - d[1:(npts - 1), 1]
  epsm = 1e-09
  if ((max(dtest) - min(dtest)) > epsm) {
    cat("\n**** ERROR: sampling interval is not uniform.\n")
    stop("**** TERMINATING NOW!")
  }
  if (padfac < 1)
    padfac = 1
  if (overlap > 50) {
    overlap = 50
  }
}
Nyq <- 1 / (2 * dt)
segpts = floor(npts / (segments - (segments - 1) * overlap / 100))
Ray <- 1 / (segpts * dt)
if ((segpts * padfac) %% 2 != 0)
  padpts = (segpts * padfac) + 1
if ((segpts * padfac) %% 2 == 0)
  padpts = (segpts * padfac)
df <- 1 / (padpts * dt)
i <- seq(1, padpts, by = 1)
freq <- df * (i - 1)
zero2Nyq = 0.5 * (padpts) + 1

ft <- rep(NA, padpts * segments)
dim(ft) <- c(padpts, segments)

ihold = 1
for (i in 1:segments) {
  dd <- data.frame(cbind(d[ihold:(ihold + segpts - 1), 1], d[ihold:(ihold + segpts - 1), 2]))
  if (demean) {
    dave <- colMeans(dd[2])
    dd[2] <- dd[2] - dave
  }
  if (detrend) {
    lm.0 <- lm(dd[, 2] ~ dd[, 1])
    dd[2] <- dd[2] - (lm.0$coeff[2] * dd[1] + lm.0$coeff[1])
  }
  if (taper) {
```

```
    w = 0.5 * (1 - cos(2 * pi * (1:segpts) / (segpts +
                                                 1)))
    dd[2] = dd[2] * w
  }
  pad <- as.numeric(dd[, 2])
  if (padpts > segpts)
    pad <- append(pad, rep(0, (padpts - segpts)))
  ft[, i] <- fft(pad)
  if (normfft)
    ft[, i] <- ft[, i] / segpts
  ihold = ihold + floor(segpts * (100 - overlap) / 100)
}
pwr <- rowMeans(Mod(ft) ^ 2)
if (verbose)
  cat("\n* Calculating Bispectrum \n")
nfreq = floor(maxF / df)
B = complex(nfreq * nfreq)
dim(B) <- c(nfreq, nfreq)
for (ii in 1:segments) {
  for (i in 1:nfreq) {
    B[i, 1:nfreq] = B[i, 1:nfreq] + ft[i, ii] * ft[1:nfreq, ii] * Conj(ft[i + (1:nfreq) - 1,
  }
}
B = B / segments
Bmag2 = Mod(B) ^ 2
if (logbis)
  Bmag3 = log(Bmag2)
if (!logbis)
  Bmag3 = Bmag2
if (verbose)
  cat("* Calculating Bicoherence: Kim and Powers method \n")
f1f2 = double(nfreq * nfreq)
dim(f1f2) <- c(nfreq, nfreq)
f3 = double(nfreq * nfreq)
dim(f3) <- c(nfreq, nfreq)
for (ii in 1:segments) {
  for (i in 1:nfreq) {
    f1f2[i, 1:nfreq] = f1f2[i, 1:nfreq] + Mod(ft[i, ii] * ft[1:nfreq, ii]) ^
      2
    f3[i, 1:nfreq] = f3[i, 1:nfreq] + Mod(ft[i + (1:nfreq) -
                                                 1, ii]) ^ 2
  }
```

```
}
f1f2 = f1f2 / segments
f3 = f3 / segments
den = double(nfreq * nfreq)
dim(den) <- c(nfreq, nfreq)
for (i in 1:nfreq) {
  den[i, 1:nfreq] = (f1f2[i, 1:nfreq] * f3[i, 1:nfreq])
}
if (CF == 0) {
  if (verbose)
    cat("\n* Calculating Conditioning Factor \n")
  p95 = sort(den)[floor(length(den) * 0.95)]
  p90 = sort(den)[floor(length(den) * 0.9)]
  p75 = sort(den)[floor(length(den) * 0.75)]
  p50 = sort(den)[floor(length(den) * 0.5)]
  p25 = sort(den)[floor(length(den) * 0.25)]
  if (verbose) {
    cat("* 95th percentile for Magnitude-squared bicoherence denominator is at",
        p95,
        "\n")
    cat("* 90th percentile for Magnitude-squared bicoherence denominator is at",
        p90,
        "\n")
    cat("* 75th percentile for Magnitude-squared bicoherence denominator is at",
        p75,
        "\n")
    cat("* 50th percentile for Magnitude-squared bicoherence denominator is at",
        p50,
        "\n")
    cat("* 25th percentile for Magnitude-squared bicoherence denominator is at",
        p25,
        "\n")
  }
  if (verbose)
    cat(
      "* Automatically assigning 75th percentile value as conditioning factor for bicoherence
    )
  CF = p75
}

den = den + CF
bic2 = double(nfreq * nfreq)
```

```r
dim(bic2) <- c(nfreq, nfreq)
bic2 = Bmag2 / den

bic2CL = double(nfreq * nfreq)
dim(bic2CL) <- c(nfreq, nfreq)
for (i in 1:nfreq) {
  for (j in 1:nfreq)
    bic2CL[i, j] = 100 * pchisq(2 * segments *
                                bic2[i, j], df = 2, lower.tail = T)
}
CLset = qchisq(CL / 100, df = 2) / (2 * segments)


##custom bicoh plot plot####
{

  #
  # dev.new(
  # title = paste("Bicoherence results"))
  layout.matrix <- matrix(c(2,0, 1,3), nrow = 2, ncol = 2,byrow=TRUE)
  graphics::layout(mat = layout.matrix,
                   heights = c(1,2), # Heights of the two rows
                   widths = c(2,1)) # Widths of the two columns

  par(mar = c(5, 5, 0, 0))

  image(
    freq[1:nfreq],
    freq[1:nfreq],
    bic2,
    col = colscale,
    useRaster = T,
    xlab = "Frequency (1/yr)",
    ylab = "Frequency (1/yr)"
  )
  contour(
    freq[1:nfreq],
    freq[1:nfreq],
    bic2,
    add = T,
    levels = CLset,
    drawlabels = F,
```

```
  lwd = 2,
  xlab = "",
  ylab = ""
)
if (is.null(id))
  abline(
    a = 0,
    b = 1,
    lwd = 4,
    col = "#FFFFFF70"
  )
if (!is.null(id)) {
  for (i in 1:length(id))
    abline(
      a = id[i],
      b = -1,
      lwd = 1,
      lty = 2
    )
}




# 50% transparent grey
trans_grey <- "#80808040"    # medium grey, alpha=80 hex
trans_dgrey <- "#A9A9A940"   # dark grey, alpha=80 hex

abline(v = periods, col = trans_grey,lwd=12)
abline(h = periods, col = trans_grey,lwd=12)


x0 <- 1/24
y0 <- 1/40

# Now add a custom 45° line starting from that point
x_end <- max(freq[1:nfreq]) * 1.2   # push to right
y_end <- y0 - (x_end - x0)          # slope = -1 (45° down)

segments(
  x0, y0,
  x_end, y_end,
```

```r
    col = "black", lwd = 3,lty=3
  )

  # mirrored 45° line across vertical axis (slope = +1)
x_end_m <- min(freq[1:nfreq]) * 0.8  # push to left
y_end_m <- y0 + (x_end_m - x0)

# segments(
#   x0, y0,
#   x_end_m, y_end_m,
#   col = "red", lwd = 3, lty = 3
# )


  x0 <- 1/40
  y0 <- 1/60

  # Now add a custom 45° line starting from that point
  x_end <- max(freq[1:nfreq]) * 1.2   # push to right
  y_end <- y0 - (x_end - x0)          # slope = -1 (45° down)

  segments(
    x0, y0,
    x_end, y_end,
    col = "black", lwd = 3,lty=3
  )

  # mirrored 45° line across vertical axis (slope = +1)
x_end_m <- min(freq[1:nfreq]) * 0.8  # push to left
y_end_m <- y0 + (x_end_m - x0)

# segments(
#   x0, y0,
#   x_end_m, y_end_m,
#   col = "red", lwd = 3, lty = 3
# )

  x0 <- 1/60
  y0 <- 1/100

  # Now add a custom 45° line starting from that point
  x_end <- max(freq[1:nfreq]) * 1.2   # push to right
```

```r
  y_end <- y0 - (x_end - x0)          # slope = -1 (45° down)

  segments(
    x0, y0,
    x_end, y_end,
    col = "black", lwd = 3,lty=3
  )

  # mirrored 45° line across vertical axis (slope = +1)
x_end_m <- min(freq[1:nfreq]) * 0.8  # push to left
y_end_m <- y0 + (x_end_m - x0)

# segments(
#   x0, y0,
#   x_end_m, y_end_m,
#   col = "red", lwd = 3, lty = 3
# )


  x0 <- 1/18
  y0 <- 1/60

  # Now add a custom 45° line starting from that point
  x_end <- max(freq[1:nfreq]) * 1.2   # push to right
  y_end <- y0 - (x_end - x0)          # slope = -1 (45° down)

  segments(
    x0, y0,
    x_end, y_end,
    col = "black", lwd = 3,lty=3
  )

  # mirrored 45° line across vertical axis (slope = +1)
x_end_m <- min(freq[1:nfreq]) * 0.8  # push to left
y_end_m <- y0 + (x_end_m - x0)

# segments(
#   x0, y0,
#   x_end_m, y_end_m,
#   col = "red", lwd = 3, lty = 3
# )
```

```r
  x0 <- 1/18
  y0 <- 1/40

  # Now add a custom 45° line starting from that point
  x_end <- max(freq[1:nfreq]) * 1.2   # push to right
  y_end <- y0 - (x_end - x0)          # slope = -1 (45° down)

  segments(
    x0, y0,
    x_end, y_end,
    col = "black", lwd = 3,lty=3
  )

  # mirrored 45° line across vertical axis (slope = +1)
x_end_m <- min(freq[1:nfreq]) * 0.8  # push to left
y_end_m <- y0 + (x_end_m - x0)

# segments(
#   x0, y0,
#   x_end_m, y_end_m,
#   col = "red", lwd = 3, lty = 3
# )

  x0 <- 1/12
  y0 <- 1/40

  # Now add a custom 45° line starting from that point
  x_end <- max(freq[1:nfreq]) * 1.2   # push to right
  y_end <- y0 - (x_end - x0)          # slope = -1 (45° down)

  segments(
    x0, y0,
    x_end, y_end,
    col = "black", lwd = 3,lty=3
  )

  # mirrored 45° line across vertical axis (slope = +1)
x_end_m <- min(freq[1:nfreq]) * 0.8  # push to left
y_end_m <- y0 + (x_end_m - x0)

# segments(
#   x0, y0,
```

```
#   x_end_m, y_end_m,
#   col = "red", lwd = 3, lty = 3
# )

    x0 <- 1/9.5
  y0 <- 1/60

  # Now add a custom 45° line starting from that point
  x_end <- max(freq[1:nfreq]) * 1.2   # push to right
  y_end <- y0 - (x_end - x0)          # slope = -1 (45° down)

  segments(
    x0, y0,
    x_end, y_end,
    col = "black", lwd = 3,lty=3
  )


  # mirrored 45° line across vertical axis (slope = +1)
x_end_m <- min(freq[1:nfreq]) * 0.8  # push to left
y_end_m <- y0 + (x_end_m - x0)

# segments(
#   x0, y0,
#   x_end_m, y_end_m,
#   col = "red", lwd = 3, lty = 3
# )
  par(mar = c(0, 5, 0.5, 0))


  plot(
    freq[1:nfreq],
    pwr[1:nfreq],
    type = "l",
    axes = TRUE,
    xaxs = "i",
    xaxt="n",
    xlab = "",
    ylab = "Power",
    lwd = 2,
    log = ply
  )
```

```r
abline(v = periods, col = trans_grey,lwd=12)

# for(i in seq_along(periods)){
#   abline(
#     b = -1,
#     a = periods[i],
#     lwd = 4,
#     col = trans_dgrey
#   )
# }
#

# text(
#   x = rep(0, length(periods)),
#   y = periods,
#   labels = round(1/periods, 3),   # show rounded values
#   pos = 4,                      # text to the right of point
#   col = "black"
# )
#
# # Plot horizontal positions
text(
  x = periods,
  y = rep(min(pwr[1:nfreq]), length(periods)),
  labels = round(1/periods, 3),
  pos = 3,                      # text above the point
  col = "black"
)



  par(mar = c(5, 0, 0, 0.5))

plot(
  pwr[1:nfreq],
  freq[1:nfreq],
  type = "l",
  axes = TRUE,
  yaxt="n",
  yaxs = "i",
  xlab = "Power",
  ylab = "",
```

```
    lwd = 2,
    log = plx
  )

  abline(h = periods, col = trans_grey,lwd=12)
  text(
    y = periods,
    x = rep(min(pwr[1:nfreq]), length(periods)),
    labels = round(1/periods, 3),
    pos = 3,                      # text above the point
    col = "black"
  )


}
```

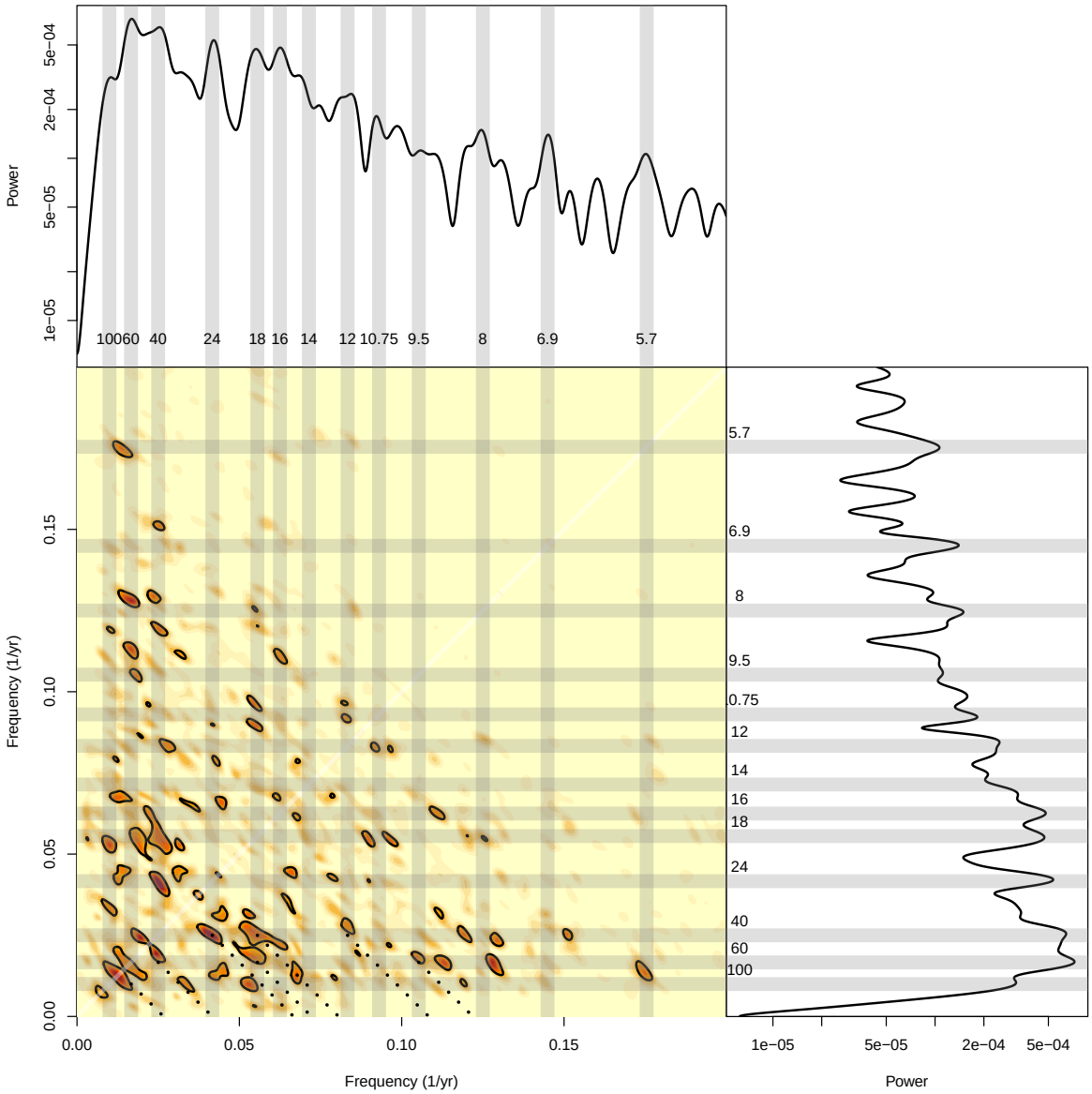

Figure 5: Figure 5. Bicoherence plot illustrating nonlinear interactions between cyclic components of the Growth Index record. The WOSA power spectra are shown along the horizontal and vertical axes, with major cyclic components indicated by gray lines. The color scale represents the bicoherence magnitude (ranging from 0 to 1) corresponding to the strength of nonlinear coupling between frequencies. Regions significant at the 95% confidence level are outlined by thick black contours, while inferred nonlinear interactions are highlighted with black dotted lines.

Example on calcualting non-linear components

```
# 2A: compute expected periods

# 60+90
# 37+24
# 7.5+60
# 7.5+37
# 9.5+60
# 37+12
# 18+37
# 18+60
# 60+37
#
#
# T1 <- 100
# T2 <- 60
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
#
# T1 <- 24
# T2 <- 40
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
#
# T1 <- 10.75
# T2 <- 40
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
```

```
#
# T1 <- 60
# T2 <- 8
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
#
# T1 <- 40
# T2 <- 8
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
#
# T1 <- 24
# T2 <- 8
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
#
# T1 <- 7
# T2 <- 40
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
#
# T1 <- 6.5
# T2 <- 60
# f1 <- 1/T1
# f2 <- 1/T2
# T_sum  <- 1/(f1 + f2)
```

```
# T_diff <- 1/abs(f1 - f2)
# cat(T1, "+", T2,"=",T_sum, "years\n")
# cat(T1, "-", T2,"=",T_diff, "years\n")
```

Non-linear component extraction

```
# ================================
# 60 + 100 → 40 yr nonlinear cycle
# WITH direct 40 yr extraction
# ================================

# -------------------------------
# Step 1. Define target periods
# -------------------------------

P1  <- 60
P2  <- 100

f1  <- 1 / P1
f2  <- 1 / P2

# -------------------------------
# Step 2. Load and prepare data
# -------------------------------

t  <- VA25mean[,1]
x  <- VA25mean[,2]

dt <- mean(diff(t))
fs <- 1 / dt


f_sum  <- f1 + f2
f_diff <- abs(f1 - f2)

T_sum  <- 1 / f_sum
T_diff <- 1 / f_diff
```

```
bw_diff <- 0.15 * f_diff
bw_sum <- 0.15 * f_sum


bw1  <- 0.15 * f1
bw2  <- 0.15 * f2

x_23  <-
taner(dat=VA25mean,padfac=2,flow=f1  - bw1,fhigh=f1  + bw1,roll=10^3,demean=T,detrend=F,addme
       output=1,xmin=0,xmax=Nyq,genplot=F,check=T,verbose=T)

x_35  <-
taner(dat=VA25mean,padfac=2,flow=f2  - bw2,fhigh=f2  + bw2,roll=10^3,demean=T,detrend=F,addme
       output=1,xmin=0,xmax=Nyq,genplot=F,check=T,verbose=T)
x_40d <-
taner(dat=VA25mean,padfac=2,flow=f_sum  - bw_sum,fhigh=f_sum  +bw_sum,roll=10^3,demean=T,det
       output=1,xmin=0,xmax=Nyq,genplot=F,check=T,verbose=T)

x_60d <-
taner(dat=VA25mean,padfac=2,flow=f_diff  - bw_diff,fhigh=f_diff  +bw_diff,roll=10^3,demean=T
       output=1,xmin=0,xmax=Nyq,genplot=F,check=T,verbose=T)


xnl <- x_23[,2] * x_35[,2]   # nonlinear 14 yr generation
xnl <- cbind(x_23[,1],xnl)

x_40nl <-
taner(
  dat = xnl,
  padfac = 2,
  flow  = f_sum - bw_sum,
  fhigh = f_sum + bw_sum,
  roll  = 10^3,
  demean = TRUE,
  detrend = FALSE,
  addmean = FALSE,
  output = 1,
  xmin = 0,
  xmax = Nyq,
  genplot = FALSE,
  check = TRUE,
  verbose = TRUE
```

```
)

#mtm(dat=xnl,tbw=3,ntap=10,padfac=5,demean=T,detrend=F,siglevel=0.9,ar1=T,output=1,
#     CLpwr=T,xmin=1/1000,xmax=1/10,pl=1,sigID=T,genplot=T,verbose=T)
```

```
# ================================
# 23 + 35 → 14 yr nonlinear cycle
# WITH direct 14 yr extraction
# ================================

# -------------------------------
# Step 1. Define target periods
# -------------------------------

P1  <- 24
P2  <- 40

f1  <- 1 / P1
f2  <- 1 / P2

# -------------------------------
# Step 2. Load and prepare data
# -------------------------------

t  <- VA25mean[,1]
x  <- VA25mean[,2]

dt <- mean(diff(t))
fs <- 1 / dt


f_sum  <- f1 + f2
f_diff <- abs(f1 - f2)

T_sum  <- 1 / f_sum
T_diff <- 1 / f_diff

bw_diff <- 0.1 * f_diff
bw_sum <- 0.1 * f_sum


bw1  <- 0.1 * f1
```

```
bw2  <- 0.1 * f2

x_24  <-
taner(dat=VA25mean,padfac=2,flow=f1  - bw1,fhigh=f1  + bw1,roll=10^3,demean=T,detrend=F,addme
       output=1,xmin=0,xmax=Nyq,genplot=FALSE,check=T,verbose=T)

x_40  <-
taner(dat=VA25mean,padfac=2,flow=f2  - bw2,fhigh=f2  + bw2,roll=10^3,demean=T,detrend=F,addme
       output=1,xmin=0,xmax=Nyq,genplot=FALSE,check=T,verbose=T)
x_15d <-
taner(dat=VA25mean,padfac=2,flow=f_sum  - bw_sum,fhigh=f_sum  +bw_sum,roll=10^3,demean=T,det
       output=1,xmin=0,xmax=Nyq,genplot=FALSE,check=T,verbose=T)


xnl <- x_24[,2] * x_40[,2]   # nonlinear 14 yr generation
xnl <- cbind(x_24[,1],xnl)

x_15nl <-
taner(
  dat = xnl,
  padfac = 2,
  flow  = f_sum - bw_sum,
  fhigh = f_sum + bw_sum,
  roll  = 10^3,
  demean = TRUE,
  detrend = FALSE,
  addmean = FALSE,
  output = 1,
  xmin = 0,
  xmax = Nyq,
  genplot = FALSE,
  check = TRUE,
  verbose = TRUE
)


# mtm(dat=xnl,tbw=3,ntap=10,padfac=5,demean=T,detrend=F,siglevel=0.9,ar1=T,output=1,
#      CLpwr=T,xmin=1/1000,xmax=1/10,pl=1,sigID=T,genplot=T,verbose=T)

# -------------------------------
# Step 6. Time-domain plots (cycles only)
# -------------------------------
```

```
#
# par(mfrow=c(4,1), mar=c(4,4,2,1))
#
# plot(t, x, type="l",
#      main="Extracted 23 yr cycle",
#      xlab="Time", ylab="Amp")
# lines(t, x_24[,2]+ mean(x),col="red")
# plot(t, x, type="l",
#      main="Extracted 35 yr cycle",
#      xlab="Time", ylab="Amp")
# lines(t, x_40[,2]+ mean(x),col="red")
#
# plot(t, x, type="l",
#      main="Directly extracted 14 yr cycle",
#      xlab="Time", ylab="Amp")
# lines(t, x_15d[,2]+ mean(x),col="red")
#
# plot(x_15nl, type="l",
#      main="Nonlinear product (23 × 35 → ~14 yr)",
#      xlab="Time", ylab="Amp")

par(mfrow=c(3,1), mar=c(4,4,2,3)) # Increased right margin for mtext

# --- Helper function for the custom BCE-CE axis ---
add_bce_ce_axis <- function(x_data) {
  # 1. Define the pretty labels you want to see
  pretty_labels <- seq(-1250, 250, by = 250)
  pretty_labels <- pretty_labels[!(pretty_labels %in% c(-1, 0, 1))]

  # 2. Map labels to plotting positions (BCE moves +1)
  pretty_at <- ifelse(pretty_labels < 0, pretty_labels + 1, pretty_labels)

  # 3. Draw rounded labels
  axis(1, at = pretty_at, labels = pretty_labels)
  # 4. Draw specific Year 1 label
  axis(1, at = 1, labels = "1", font = 2)
}

# (1) Original + cycles
plot(t, x, type="l",
     main="Nonlinear cycles",
     xlab="", ylab="GI", xaxt="n")
```

```r
lines(x_15d[,1], x_15d[,2] + mean(x), col="green")
lines(x_40d[,1], x_40d[,2] + mean(x), col="red")
add_bce_ce_axis(t)


# --- 40 kyr plot ---
plot(x_40d[,1], x_40d[,2] + mean(x),
     lwd=1, type="l", col="red",
     xlab="", ylab="GI", xaxt="n")
add_bce_ce_axis(x_40d[,1])

par(new=TRUE)
plot(x_40nl[,1], x_40nl[,2],
     lwd=1, col="black", type="l",
     axes=FALSE, xlab="", ylab="")
axis(4)  # right y-axis
mtext("Nonlinear combination", side=4, line=2.5, cex=0.8)


# --- 15 kyr plot ---
plot(x_15d[,1], x_15d[,2] + mean(x),
     lwd=1, type="l", col="green",
     xlab="Year", ylab="GI", xaxt="n")
add_bce_ce_axis(x_15d[,1])

par(new=TRUE)
plot(x_15nl[,1], x_15nl[,2],
     lwd=1, col="black", type="l",
     axes=FALSE, xlab="", ylab="")
axis(4)  # right y-axis
mtext("Nonlinear combination", side=4, line=2.5, cex=0.8)
```

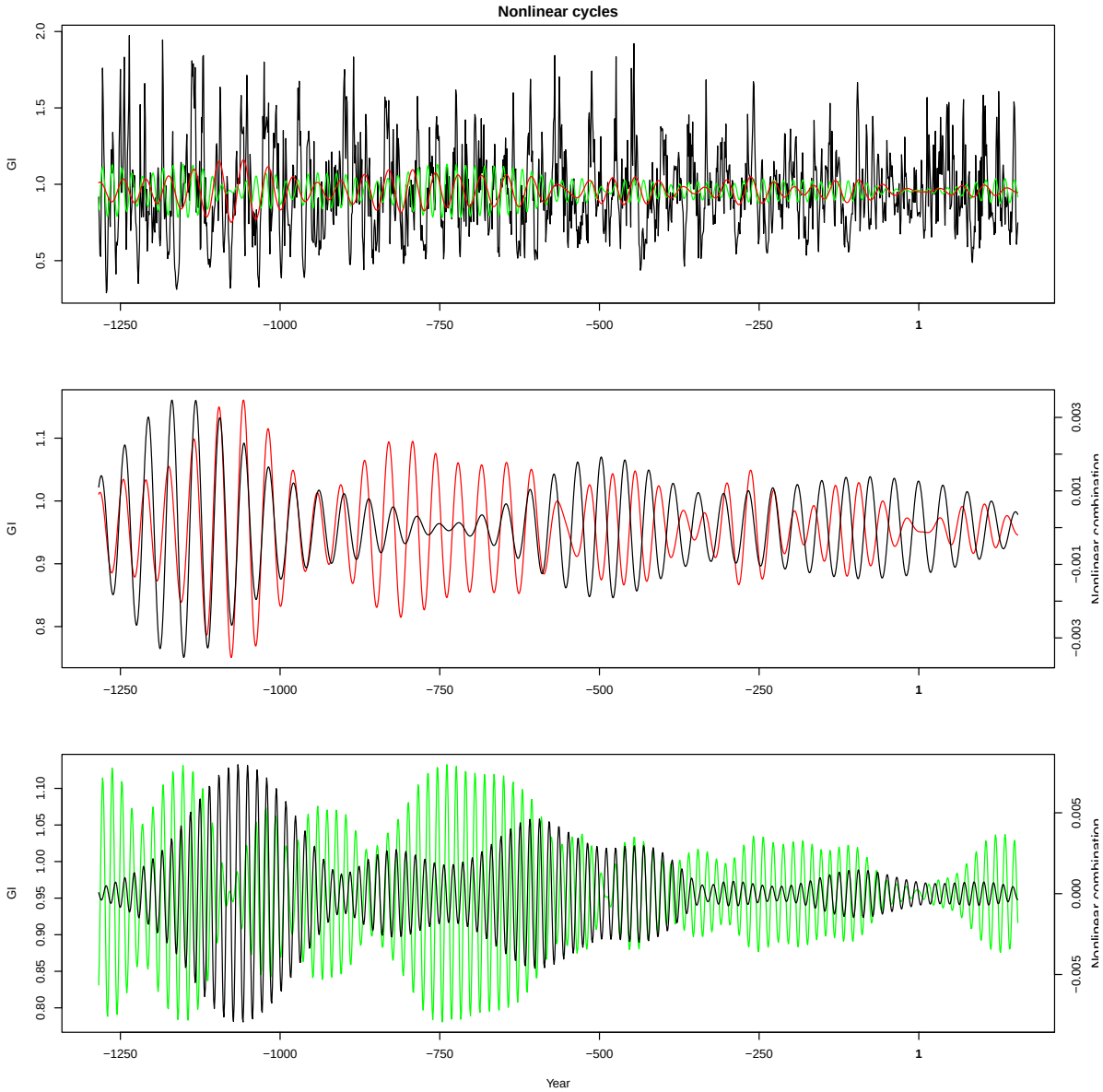

Figure 6: Figure

Load the stalagmite record of Baker et al., 2015

```
stal_scot <- read.csv("https://raw.githubusercontent.com/stratigraphy/Weesp-dendro-cycles/mai
#stal_scot <- read.csv("stal_scot.csv")
```

```
stal_scot <- stal_scot[,c(2,3)]
stal_scot <- sortNave(stal_scot,genplot=FALSE,verbose=FALSE)
stal_scot[,2] <-scale(stal_scot[,2])

stal_scot_shift <- stal_scot
stal_scot_shift[,1] <- stal_scot_shift[,1]
stal_scot_shift <- linterp(stal_scot_shift,genplot=FALSE,verbose = FALSE)
```

Cross Wavelet analysis

```
#The analyze_xwavelet function is a modified version of the analyze_Xwavelet function from th
analyze_xwavelet <- function(
    data_1,
    data_2,
    dj = 1/100,
    lowerPeriod = 2,
    upperPeriod = 1024,
    verbose = FALSE,
    omega_nr = 4
) {

  # --- PREPROCESSING ---
  data_1 <- data_1[order(data_1[,1]), ]
  data_2 <- data_2[order(data_2[,1]), ]

  xmin <- max(min(data_1[,1]), min(data_2[,1]))
  xmax <- min(max(data_1[,1]), max(data_2[,1]))

  dx <- max(median(diff(data_1[,1])), median(diff(data_2[,1])))
  grid <- seq(xmin, xmax, by = dx)

  d1 <- data.frame(
    x = grid,
    y = approx(data_1[,1], data_1[,2], grid, rule = 1)$y
  )
  d2 <- data.frame(
    x = grid,
    y = approx(data_2[,1], data_2[,2], grid, rule = 1)$y
  )
```

```r
# --- WAVELETS ---
wt1 <- analyze_wavelet(
  d1,
  dj=dj,
  lowerPeriod = lowerPeriod,
  upperPeriod = upperPeriod,
  omega_nr = omega_nr,
  verbose = verbose,
  pval = FALSE,
  n_simulations = 10,
  run_multicore = FALSE
)

wt2 <- analyze_wavelet(
  d2,
  dj=dj,
  lowerPeriod = lowerPeriod,
  upperPeriod = upperPeriod,
  omega_nr = omega_nr,
  verbose = verbose,
  pval = FALSE,
  n_simulations = 10,
  run_multicore = FALSE
)

nr <- wt1$nr
nc <- wt1$nc
Scale <- wt1$Scale

# --- CROSS-WAVELET ---
Wave.xy <- (wt1$Wave * Conj(wt2$Wave)) /
  matrix(rep(Scale, nc), nrow = nr)

Power.xy <- Mod(Wave.xy)
Phase.xy <- Arg(Wave.xy)

Power.avg = rowMeans(as.matrix(Power.xy))



out <- list(
  Wave   = t(Wave.xy),
```

```
    Power  = t(Power.xy),
    Phase  = t(Phase.xy),
    Power.avg = Power.avg,
    dt = dt,
    dj = dj,
    Scale = Scale,
    Period = wt1$Period,
    nc = nc,
    nr = nr,
    axis.1 = wt1$axis.1,
    axis.2 = wt1$axis.2,
    omega_nr = omega_nr,
    x1 = wt1$x,
    y1 = wt1$y,
    x2 = wt2$x,
    y2 = wt2$y
  )

  class(out) <- "analyze.Xwavelet"
  return(out)
}

x_wt<- analyze_xwavelet(
  data_1=cbind(stal_scot_shift[,1],scale(stal_scot_shift[,2])),
  data_2=cbind(VA25mean[,1],scale(VA25mean[,2])),
  dj = 1/100,
  lowerPeriod = 1.5,
  upperPeriod = 128+64,
  verbose = FALSE,
  omega_nr = 15
)


add_bce_ce_axis <- function() {
  # 1. Define the labels
  pretty_labels <- seq(-2000, 2000, by = 250)
  pretty_labels <- pretty_labels[!(pretty_labels %in% c(-1, 0, 1))]

  # 2. Map labels to plotting positions (BCE moves +1)
  pretty_at <- ifelse(pretty_labels < 0, pretty_labels + 1, pretty_labels)

  # 3. Draw labels
```

```
  axis(1, at = pretty_at, labels = pretty_labels)
  # 4. Draw specific Year 1 label
  axis(1, at = 1, labels = "1", font = 2)
}

#The plot_Xwavelet function is a modified version of the plot_Xwavelet function from the Wave


plot_Xwavelet <- function(Xwavelet = NULL, lowerPeriod = NULL, upperPeriod = NULL,
                          n.levels = 100, palette_name = "rainbow", color_brewer = "grDevice
                          useRaster = TRUE, periodlab = "Period (metres)", x_lab = "depth (m
                          keep_editable = FALSE, dev_new = TRUE, plot_dir = TRUE,
                          plot_arrows = TRUE,
                          pwr_quant = 0.85,
                          n_arrows = 75,
                          T_unit = 15,
                          P_unit  = 0.5,
                          arrow_length = 0.075,
                          arrow_lwd = 1.9,
                          arrow_angle = 25, add_lines = NULL, add_points = NULL, add_abline_
                          add_abline_v = NULL, add_data = TRUE, add_avg = FALSE, plot_horizo

  if (keep_editable == FALSE) {
    oldpar <- par(no.readonly = TRUE)
    on.exit(par(oldpar))
  }

  Power <- t(Xwavelet$Power)
  Phase <- t(Xwavelet$Phase)
  maximum.level <- max(Xwavelet$Power, na.rm = TRUE)
  power_max_mat.levels <- quantile(Xwavelet$Power, probs = seq(from = 0, to = 1, length.out =

  # Color Palette Logic
  if (color_brewer == "RColorBrewer") {
    key.cols <- rev(colorRampPalette(RColorBrewer::brewer.pal(RColorBrewer::brewer.pal.info[p
  }
  if (color_brewer == "colorRamps") {
    color_brewer_Sel <- paste("colorRamps::", palette_name, "(n=n.levels)")
    key.cols <- eval(parse(text = color_brewer_Sel))
  }
  if (color_brewer == "grDevices") {
    if (palette_name == "rainbow") {
```

```
    color_brewer_Sel <- "grDevices::rainbow(n=n.levels, start = 0, end = 0.7)"
    key.cols <- rev(eval(parse(text = color_brewer_Sel)))
  } else if (palette_name %in% c("heat.colors", "terrain.colors", "topo.colors", "cm.colors
    color_brewer_Sel <- paste("grDevices::", palette_name, "(n=n.levels, start = 0, end = 
    key.cols <- rev(eval(parse(text = color_brewer_Sel)))
  } else {
    key.cols <- hcl.colors(n = n.levels, palette = palette_name, alpha = NULL, rev = FALSE
  }
}
if (color_brewer == "scico") {
  color_brewer_Sel <- paste("scico::scico(n = n.levels, palette = '", palette_name, "', di
  key.cols <- rev(eval(parse(text = color_brewer_Sel)))
}
if (color_brewer == "viridis") {
  color_brewer_Sel <- paste("viridis::", palette_name, "(n=n.levels, direction = -1)")
  key.cols <- rev(eval(parse(text = color_brewer_Sel)))
}

periodtck <- 0.02
periodtcl <- 0.5
lwd.axis <- 1
legend.params <- list(width = 1.2, shrink = 0.9, mar = 5.1, n.ticks = 6, label.digits = 3,
key.marks <- round(seq(from = 0, to = 1, length.out = legend.params$n.ticks) * n.levels)
key.labels <- formatC(as.numeric(power_max_mat.levels), digits = legend.params$label.digits

if (dev_new == TRUE) {
  if (plot_horizontal == TRUE) dev.new(width = 15, height = 7, noRStudioGD = TRUE)
  else dev.new(width = 7, height = 10, noRStudioGD = TRUE)
}

depth <- as.numeric(Xwavelet$x1)
if (plot_dir != TRUE) xlim_vals <- rev(range(depth))
else xlim_vals <- range(depth)

lowerPeriod <- if (is.null(lowerPeriod)) min(Xwavelet$Period) else lowerPeriod
upperPeriod <- if (is.null(upperPeriod)) max(Xwavelet$Period) else upperPeriod
ylim_vals <- c(lowerPeriod, upperPeriod)

if (add_data == TRUE && add_avg == TRUE && plot_horizontal == TRUE) {
  layout.matrix <- matrix(c(1, 2, 4, 3), nrow = 2, ncol = 2, byrow = TRUE)
  graphics::layout(mat = layout.matrix, heights = c(0.25, 1), widths = c(8, 2))
```

```
# --- Panel 1: Proxy Data ---
par(mar = c(0, 4, 2, 0))
plot(y = Xwavelet$y1, x = Xwavelet$x1, type = "l", col = "black", xaxs = "i",
     xlim = xlim_vals, ylab = "proxy_1", xlab = "", yaxt = "s", xaxt = "n")
par(new = TRUE)
plot(y = Xwavelet$y2, x = Xwavelet$x2, type = "l", col = "red", xaxs = "i",
     xlim = xlim_vals, ylab = "", xlab = "", yaxt = "n", xaxt = "n")
axis(side = 4, col = "red", col.axis = "red")
mtext("proxy_2", side = 4, line = 2, col = "red")
add_bce_ce_axis() # Custom X-axis for proxy panel

if (!is.null(add_abline_v)) abline(v = add_abline_v)

# --- Panel 2: Color Key ---
par(new = FALSE, mar = c(3, 2, 2, 2), mgp = c(2, 1, 0))
image(x = seq(0, n.levels), y = 1, z = t(matrix(power_max_mat.levels, nrow = 1)),
      col = key.cols, breaks = power_max_mat.levels, useRaster = TRUE, yaxt = "n", xaxt =
      xlab = "Power", ylab = "")
axis(1, lwd = lwd.axis, at = key.marks, labels = NA, tck = 0.02, tcl = 1.2)
mtext(key.labels, side = 1, at = key.marks, line = 0.1, las = 2, cex = 0.75)
box(lwd = lwd.axis)

# --- Panel 3: Average Power ---
par(new = FALSE, mar = c(4, 0, 0, 0.5))
plot(x = Xwavelet$Power.avg, y = Xwavelet$Period, log = "y", type = "l",
     yaxs = "i", xlab = "Wt. power", xaxs = "i", ylim = sort(ylim_vals))
if (!is.null(add_abline_h)) abline(h = add_abline_h)

# --- Panel 4: Wavelet Power Image ---
par(new = FALSE, mar = c(4, 4, 0, 0), xpd = FALSE)
image(x = Xwavelet$x1, y = Xwavelet$axis.2, z = Xwavelet$Power,
      col = key.cols, breaks = power_max_mat.levels, useRaster = TRUE,
      ylab = periodlab, xlab = x_lab, axes = TRUE, yaxt = "n", xaxt = "n",
      xlim = xlim_vals, ylim = log2(sort(ylim_vals)))
box(lwd = lwd.axis)

# Add custom X-axis to the heatmap
add_bce_ce_axis()

# Y-axis (Period) logic
period.tick <- unique(trunc(Xwavelet$axis.2))
period.tick <- period.tick[period.tick >= min(log2(ylim_vals)) & period.tick <= max(log2
```

```
    axis(2, lwd = lwd.axis, at = period.tick, labels = NA, tck = periodtck, tcl = periodtcl)
    axis(4, lwd = lwd.axis, at = period.tick, labels = NA, tck = periodtck, tcl = periodtcl)
    mtext(2^period.tick, side = 2, at = period.tick, las = 2, line = par()$mgp[2] - 0.5,
          font = par()$font.axis, cex = par()$cex.axis)

    if (!is.null(add_lines)) {
      for (i in 2:ncol(add_lines)) lines(add_lines[, 1], log2(add_lines[, i]))
    }
    if (!is.null(add_points)) {
      for (i in 2:ncol(add_points)) points(add_points[, 1], log2(add_points[, i]))
    }
    if (!is.null(add_abline_h)) abline(h = log2(add_abline_h))
    if (!is.null(add_abline_v)) abline(v = add_abline_v)

    # Phase Arrows logic
    if (plot_arrows == TRUE) {
      maxdetect <- matrix(0, nrow = Xwavelet$nr, ncol = Xwavelet$nc)
      for (j in 1:Xwavelet$nc) {
        for (i in 2:(Xwavelet$nr - 1)) {
          if ((Power[i, j] > Power[i + 1, j]) && (Power[i, j] > Power[i - 1, j])) maxdetect[
        }
      }
      p_thresh <- quantile(Xwavelet$Power, pwr_quant)
      t_idx <- seq(1, Xwavelet$nc, by = round(Xwavelet$nc / n_arrows))
      s_idx <- 1:Xwavelet$nr

      for (i in t_idx) {
        for (j in s_idx) {
          if (Power[j, i] >= p_thresh && maxdetect[j, i] == 1) {
            x0 <- Xwavelet$x1[i]; y0 <- Xwavelet$axis.2[j]; angle <- Phase[j, i]
            x1 <- x0 + cos(angle) * T_unit; y1 <- y0 + sin(angle) * P_unit
            arrows(x0, y0, x1, y1, length = arrow_length, lwd = arrow_lwd, angle = arrow_angl
          }
        }
      }
    }
  }
}

# --- Usage ---
# Analyze data
x_wt <- analyze_xwavelet(
```

```r
  data_1 = cbind(stal_scot_shift[, 1], scale(stal_scot_shift[, 2])),
  data_2 = cbind(VA25mean[, 1], scale(VA25mean[, 2])),
  dj = 1/100,
  lowerPeriod = 1.5,
  upperPeriod = 128 + 64,
  verbose = FALSE,
  omega_nr = 15
)

# Plot with custom axis integrated
plot_Xwavelet(Xwavelet = x_wt, dev_new = FALSE, P_unit = 0.2, pwr_quant = 0.8,
              keep_editable = TRUE, add_avg = TRUE, x_lab = "Year (BCE/CE)")
```

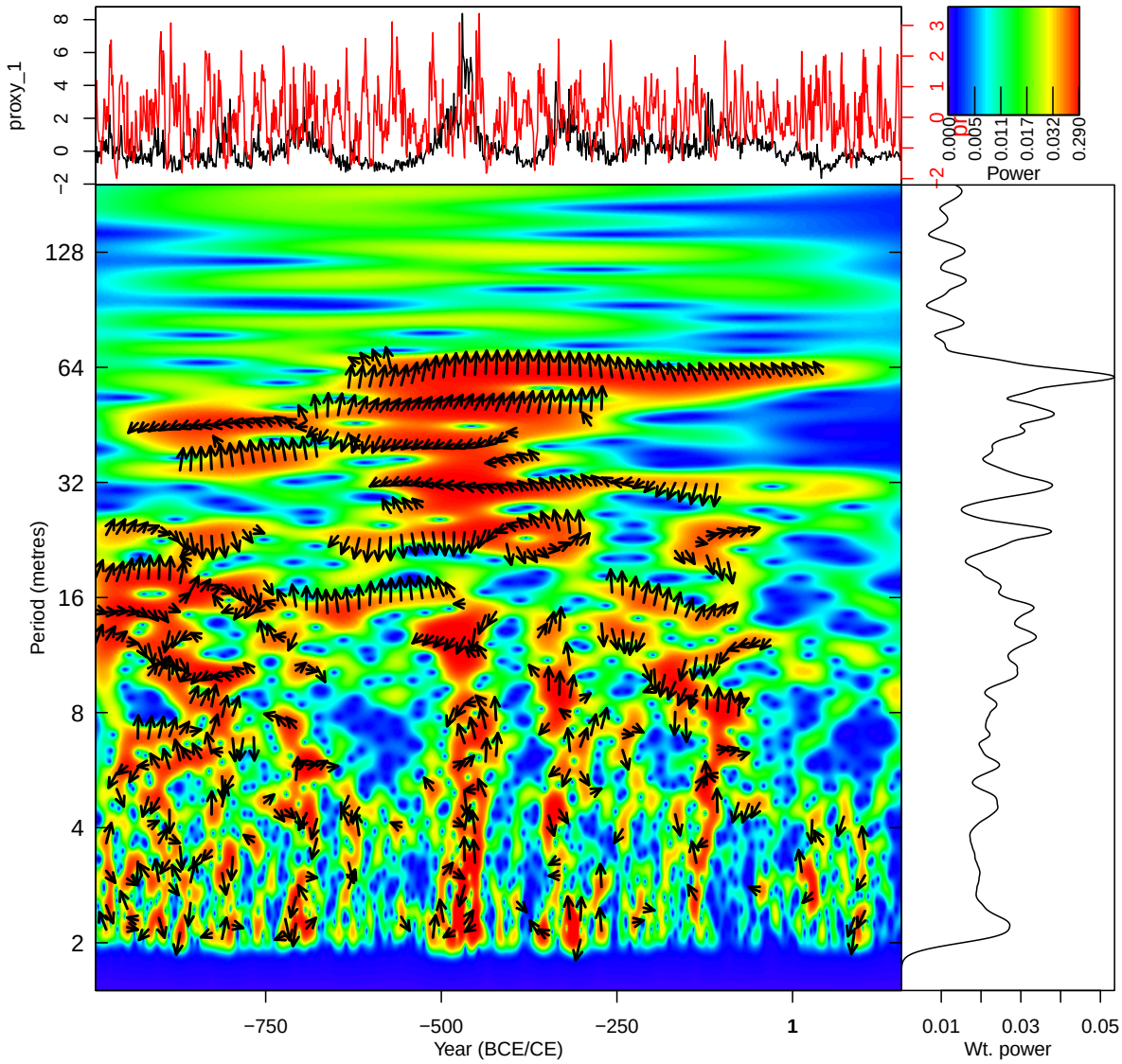

Figure 7: Figure 7.

```
####synthetic check ####

# 1. Setup Time and Reference Signal (Data 1)
# Time from 0 (Oldest) to 20 (Youngest)
t <- seq(100, 200, by = 0.1)
period <- 5
```

```r
# Reference Signal (Data 1) - Black Solid Line
d1 <- sin(2 * pi * t / period)

# 2. Create the scenarios for Data 2 - Red Dashed Line
# Scenario A: In-Phase (0 degrees) -> Right Arrow
d2_inphase <- sin(2 * pi * t / period)

# Scenario B: Anti-Phase (180 degrees) -> Left Arrow
d2_antiphase <- sin(2 * pi * t / period + pi)

# Scenario C: Data 1 leads Data 2 by 90 degrees -> Up Arrow
# We shift Data 2 to the right (delay it) by 1/4 of the period
d2_lag <- sin(2 * pi * (t - 1.25) / period)

# 3. Visual Verification Plots
graphics.off()
par(mfrow = c(3, 1), mar = c(4, 4, 2, 1))

# --- PLOT 1: IN-PHASE ---
plot(t, d1, type = "l", lwd = 2, col = "black",
     main = "In-Phase (Arrow ->)", xlab = "Time (Oldest to Youngest)", ylab = "Value")
lines(t, d2_inphase, col = "red", lty = 2, lwd = 2)
legend("topright", legend=c("Data 1 (Ref)", "Data 2"), col=c("black", "red"), lty=c(1, 2), b

# --- PLOT 2: ANTI-PHASE ---
plot(t, d1, type = "l", lwd = 2, col = "black",
     main = "Anti-Phase (Arrow <-)", xlab = "Time (Oldest to Youngest)", ylab = "Value")
lines(t, d2_antiphase, col = "red", lty = 2, lwd = 2)

# --- PLOT 3: DATA 1 LEADS DATA 2 (90 deg) ---
plot(t, d1, type = "l", lwd = 2, col = "black",
     main = "Data 1 Leads Data 2 (Arrow ^)", xlab = "Time (Oldest to Youngest)", ylab = "Valu
lines(t, d2_lag, col = "red", lty = 2, lwd = 2)
# Note: The Black peak is to the LEFT of the Red peak.
# It happened earlier in time.

x_wt<- analyze_xwavelet(
  data_1=cbind(t,d1),
  data_2=cbind(t,d2_lag),
  dj = 1/100,
  lowerPeriod = 1.5,
  upperPeriod = 128+64,
```

```
  verbose = FALSE,
  omega_nr = 15
)
#?plot_Xwavelet

plot_Xwavelet(Xwavelet = x_wt,dev_new = FALSE, P_unit = 0.2,  pwr_quant = 0.8,
  keep_editable = TRUE,add_avg = TRUE)
```